

DevOps

Terminal and VCS

*Tomislav Nikic, MSc
University of Applied Sciences Campus Vienna
WS 2025/2026*

Why Learn the Terminal & Git?

- **Terminal:** Essential for scripting, automation, and controlling systems efficiently.
- **Git:** Industry-standard version control system used by individuals and teams worldwide.
- **Purpose:** Empower you to work faster, smarter, and with better collaboration.

What Is a Shell?

- A **shell** is a command-line interpreter that provides a user interface for accessing OS services.
- It allows users to type commands to run programs, move files, and more.
- Common types: sh, bash, zsh, fish.
- It sits between the user and the kernel, translating commands into system calls.
- Types:
 - **Login Shell:** Started when you log in.
 - **Interactive Shell:** For command-line use.

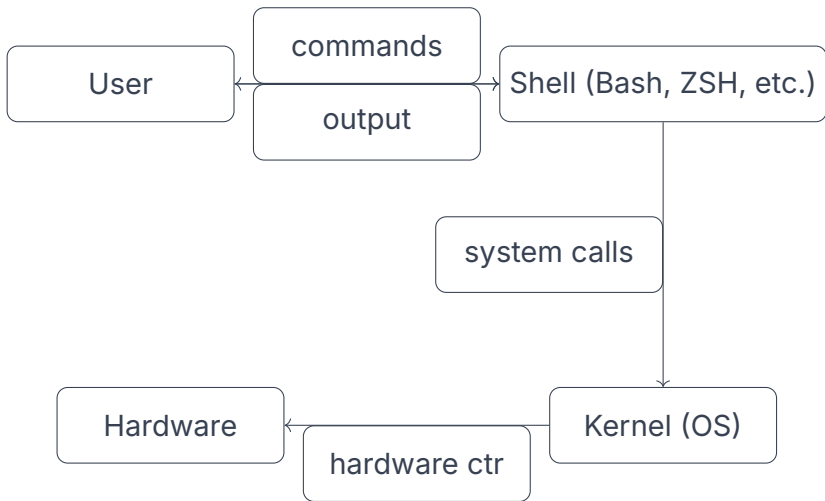
Understanding the Terminal

- A **Command Line Interface (CLI)** enables interacting with the system via text commands.
- Compared to GUIs: Offers more control, automation, and speed.
- Originated with Unix systems and standardized via POSIX.

Bash vs ZSH

- **Bash (Bourne Again SHell):**
 - Default on many Linux distros.
 - Mature and widely supported.
- **ZSH (Z Shell):**
 - Enhanced features like tab-completion and themes.
 - Supports plugins (e.g., Oh My Zsh).
- Choose ZSH for customization and modern features.

Shell Interaction



Shell Scripting - Introduction

- Scripts automate tasks like backups or deployments.
- A basic script:

```
#!/bin/bash  
echo "Hello, world!"
```

- Use variables, loops, conditionals to add logic.

Scripting - Control Structures

- Variables: name="Alice"
- Conditionals:

```
if [ "$name" == "Alice" ]; then  
    echo "Hi Alice"  
fi
```

- Loops:

```
for i in 1 2 3 4 5  
do  
    echo "Welcome $i times"  
done
```

Editing in the Terminal

- **Nano:**
 - Easy to use.
 - Ctrl-based commands shown at bottom.
- **Vim:**
 - Modal editor: Normal, Insert, Visual, Command.
 - Useful commands: :w, :q, dd, /search.
 - Steep learning curve, but very powerful.

Monitoring System Processes

- `ps aux` : Snapshot of current processes.
- `top` / `htop` : Real-time process viewer (`htop` is interactive).
- `kill` , `killall` : Terminate processes.
- `nice` / `renice` : Adjust process priority.
- Useful for diagnosing resource-hungry tasks.

Performance Monitoring Tools

- `uptime` : System load average.
- `free` : Memory stats.
- `vmstat` , `iostat` : Performance diagnostics.
- `iotop` : Disk usage per process.
- Use to identify CPU, memory, or disk bottlenecks.

Networking Tools

- `ping`, `tracert`: Connectivity and route testing.
- `netstat`, `ss`: Show open connections and listening ports.
- `dig`, `nslookup`: DNS queries.
- `curl`, `wget`: Fetch data from URLs (great for APIs).

Text Manipulation Utilities

- View content: `cat`, `less`, `more`.
- Search: `grep`.
- Transform: `awk`, `sed`, `cut`.
- Analyze: `sort`, `uniq`, `wc`, `tr`.
- Use these to clean and extract data in shell scripts.

sed: Quick In-Place Config Change

```
sed -i 's/^PORT=.* /PORT=8081/' .env
```

Rewrites the line beginning with PORT= to new value.

Processes in the Background

- Add `&` at the end of a command to run it in the background:
 - `sleep 30 &`
- View background jobs: `jobs`
- Bring a job to the foreground: `fg %1`
- Suspend a running process: `Ctrl+Z`, then resume in background with `bg`
- Useful for multitasking and running long processes without blocking your terminal.

Managing Jobs

- List jobs: `jobs`
- Move job to background: `bg %job_number`
- Move job to foreground: `fg %job_number`
- Kill a background process: `kill %job_number` or `kill PID`
- Example workflow:
 - `python myscript.py &` (run in background)
 - `jobs` (find job number)
 - `fg %1` (bring to foreground)

nohup and Session Hangups

- On logout or SSH disconnect, the shell sends **SIGHUP** to child processes. Many programs exit on SIGHUP.
- **nohup** runs a command immune to SIGHUP and detaches standard I/O:
 - Ignores SIGHUP.
 - Redirects `stdin` to `/dev/null`, `stdout/stderr` to `nohup.out` (unless you redirect).
- Typical usage to keep a process alive after logout:

```
nohup ./webapp > webapp.log 2>&1 &
```

- Use `&` to put it in the background; check with `ps`, `pgrep`, or `ss -ltn`.

Logging Output and Errors

- Standard output (`stdout`) and error (`stderr`) can be redirected:
 - `ls > output.txt` (stdout to file)
 - `ls notfound 2> error.txt` (stderr to file)
 - `command > out.txt 2> err.txt` (split output and errors)
 - `command > all.txt 2>\&1` (combine stdout and stderr)
- Useful for debugging and keeping logs.

Piping and Redirecting

- Use the pipe (`|`) to pass output of one command as input to another:
 - `cat log.txt | grep "ERROR"`
 - `ps aux | grep python`
- Chain multiple commands for powerful workflows.
- Combine pipes with output redirection for flexible logging and filtering.

What is SSH?

- **SSH (Secure Shell):** protocol for securely connecting to remote computers.
- Encrypts communication for remote commands, file transfer, and tunneling.
- Popular for remote server administration and automation.

How Does SSH Work?

- Uses public-key cryptography to establish a secure, encrypted connection.
- Client and server negotiate encryption and authentication methods.
- Authentication can be via password or cryptographic key pair.
- Once connected, all traffic is encrypted between client and server.
- Protects against eavesdropping, man-in-the-middle attacks, and credential theft.

Basic SSH Usage

- Connect to a remote server:

```
ssh username@host
```

- Example: `ssh alice@192.168.1.5`
- Prompts for password or uses SSH keys for authentication.

SSH Keys

- Generate a key pair:

```
ssh-keygen -t ed25519 -C "you@mail.com"
```

- Public key: `~/.ssh/id_ed25519.pub`
- Private key: `~/.ssh/id_ed25519`
- Copy your public key to the server for passwordless login: `ssh-copy-id username@host`

Copying Files with SCP

- Copy file to a remote server:

```
scp file.txt username@host:/remote/path/
```

- Copy file from a remote server:

```
scp username@host:/remote/path/file.txt .
```

Port Forwarding (Tunneling)

- Forward your local port to a remote server:

```
ssh -L 8080:localhost:80 username@host
```

- Example: Access remote web server on your local port 8080.
- Useful for securely accessing services behind a firewall.

Useful SSH Options

- Run a command remotely:

```
ssh username@host "uptime"
```

- Use a specific identity file:

```
ssh -i ~/.ssh/my_key username@host
```

- Enable agent forwarding: `ssh -A username@host`

- Increase verbosity for debugging:

```
ssh -v username@host
```

Filesystem Links: Why They Matter

- Use links to point a stable name to changing versions (e.g., current release).
- Two kinds:
 - **Hard link**: another name for the same inode (data).
 - **Symbolic link**: a path pointer to a target.
- In deployments: flip a symlink to swap versions quickly and atomically.

Hard Links: Concept & Constraints

- Share the same inode as the original file (same data).
- Constraints:
 - Same filesystem only.
 - Typically cannot link directories.
 - Deleting one name does not remove data until all hard links are gone.
- Good for: duplicate names for immutable files on the same volume.

Hard Links: Basic Commands

```
# Create a hard link
ln original.txt hardlink.txt

# Show shared inode numbers (should match)
ls -li original.txt hardlink.txt
```

Symbolic Links: Concept & Behavior

- A small file containing a path to a target (file or directory).
- Can cross filesystems; can point to directories.
- Can become *broken* if the target path disappears or moves.
- Path resolution follows the link at access time.

Symbolic Links: Basic Commands

```
# Create a symlink (absolute or relative target)
ln -s target_path symlink_name

# Remove a symlink (does not delete target)
rm symlink_name
```

Managing Symlinks Safely

- Prefer **relative** symlinks inside a tree for portability.
- Update atomically with `-sfn` (create/force/replace without error):

```
ln -sfn ~/webapp_versions/20250804_102015  
↪ ~/webapp_current
```

Symlinks for Deployments: Version Switching

- Layout:
 - `/webapp_versions/<timestamp>/` (release dirs)
 - `/webapp_current` → symlink to current release
- Steps:
 - Deploy to new timestamped dir.
 - Flip `webapp_current` symlink to new dir.
 - Restart process if needed.
- Benefit: rollback = repoint symlink to previous version.

What is Git?

- Git is a **distributed version control system (DVCS)**.
- Created by Linus Torvalds to manage Linux kernel development.
- Tracks changes and supports collaboration.
- Local and remote repositories enable flexibility.

Git Under the Hood

- Git stores **snapshots**, not diffs.
- Uses SHA-1 hashes for identifying objects.
- Object types: blobs, trees, commits, tags.
- The `.git` folder contains all repo data and history.

Git Basics

- Initialize: `git init`
- Stage files: `git add`
- Commit: `git commit -m "message"`
- Status: `git status`
- Log: `git log`

Branching in Git

- Branches are pointers to commits.
- Common commands:
 - `git branch dev`
 - `git checkout dev`
 - `git merge dev`
- Enables parallel work and experimentation.
- Visualize with `git log --graph`.

Remotes and Collaboration

- Remote = hosted version (e.g., GitHub).
- Commands:
 - `git remote add origin URL`
 - `git push`, `git pull`, `git fetch`
 - `git clone URL`
- Share your code and collaborate via remotes.

Undoing Mistakes in Git

- Restore: `git restore`, `git checkout`
- Unstage: `git reset HEAD file`
- Revert commits: `git revert <hash>`
- `git reset --hard` is destructive!
- Use `git reflog` to recover lost commits.

Viewing Changes (Diffs)

- Unstaged: `git diff`
- Staged: `git diff --staged`
- Between commits: `git diff commit1 commit2`
- Use diffs for debugging and reviewing changes.

Rewriting Git History

- Amend last commit: `git commit --amend`
- Interactive rebase: `git rebase -i HEAD 3`
- Rewriting history is dangerous on shared branches.
- Clean up commit history before merging.

Tagging in Git

- Mark key points in history, e.g., releases.
- Create:
 - Lightweight: `git tag v1.0`
 - Annotated: `git tag -a v1.0 -m "message"`
- Push tags: `git push origin v1.0`
- Use tags to organize versions.

Git Hooks

- Scripts that run at certain Git events.
- Types:
 - Pre-commit: test/lint
 - Post-merge: notify tools
- Stored in `.git/hooks/`
- Can enforce standards or run checks automatically.

Submodules in Git

- Include another Git repo inside yours.
- Use for libraries or dependencies.
- Commands:
 - `git submodule add URL`
 - `git submodule update --init`
 - `git clone --recursive`
- Adds complexity—use only when needed.

Why Git Best Practices Matter

- Clean history accelerates onboarding, reviews, debugging.
- Enables automated pipelines (CI/CD) to trust structure.
- Reduces merge friction and production risk.

Commit Message Guidelines

- Use imperative, concise subject (max 50 chars).
- Separate subject and body with a blank line.
- Explain the WHY in the body; WHAT is visible in diff.
- Reference issues/tickets: Fixes #123
- Example (Conventional Commits):

```
feat(auth): add JWT refresh token rotation
```

```
Implements rotation to reduce replay risk.  
Ref: SEC-42
```

Branch Naming Conventions

- Pattern: `<type>/<short-topic>`, e.g.:
 - `feat/payment-webhook`
 - `fix/race-on-shutdown`
 - `chore/deps-2025-09`
- Benefits:
 - Easier filtering in UI.
 - Automatable rules (CI per type).
- Keep branches short-lived to limit divergence.

Branching Strategies Compared

- Trunk-Based:
 - Small, frequent merges to main.
 - Feature flags for incomplete work.
- Git Flow:
 - Separate develop, release, hotfix branches.
 - Can be heavy; good for slower cadence.
- GitHub Flow:
 - Branch → PR → merge → deploy.
 - Suits continuous delivery.
- Choose simplest model that fits release cadence + compliance.

Rebase vs Merge

- Merge:
 - Preserves true chronology.
 - Can create noisy merge commits.
- Rebase (before PR merge):
 - Linear history, easier bisect.
 - Rewrites commits; avoid on shared branches.
- Guideline: rebase local feature branches onto updated main; merge (squash) into main.

Squash vs Merge Commit

- Squash for small/iterative fix commits → one clean commit:

```
git merge --squash feature/rotating-tokens
```

- Retain individual commits when each is meaningful (e.g. large refactor steps).

.gitignore and Repository Hygiene

- Exclude build artifacts, secrets, local editor files.
- Keep .gitignore minimal + purposeful; no mass blanket ignores.
- Periodically clean stale ignored files from repo history (if leaked).

Summary and Best Practices

- Mastering the terminal increases efficiency and automation.
- Git is essential for modern development and collaboration.
- Tips:
 - Commit early and often.
 - Use clear messages.
 - Avoid force-pushing shared branches.
 - Clean up history with rebase before merging.

Further Reading & Resources

- **Terminal:**
 - <https://tldr.sh>
 - <https://explainshell.com>
- **Git:**
 - <https://git-scm.com/book>
 - <https://learngitbranching.js.org>
- **Vim:**
 - <https://vim-adventures.com>
 - Run `vimtutor` in terminal

Q & A

Questions?